Práctica 2

Manuel Suano Gallardo

Descripción del corpus

El corpus está compuesto por 7 carpetas, las cuales representan cada una clase diferente. Para cada una de estas clases, tenemos que la media de palabras que tienen los documentos, así como la desviación típica. También se aportan datos del cuál es el mínimo y máximo de palabras que tienen un documento en la clase:

- Clase comp.sys.ibm.pc.hardware:
  - Mínimo: 48
  - Máximo: 4159
  - Media: 248.7
  - Desviación típica: 373.05
- Clase comp.sys.mac.hardware:
  - Mínimo: 29
  - Máximo: 723
  - Media: 185.38
  - Desviación típica: 108.42
- Clase rec.autos:
  - Mínimo: 31
  - Máximo: 853
  - Media: 217.21
  - Desviación típica: 170.52
- Clase rec.sport.hockey:
  - Mínimo: 33
  - Máximo: 1570
  - Media: 274.88
  - Desviación típica: 265.68
- Clase sci.electronics:
  - Mínimo: 48
  - Máximo: 11840
  - Media: 266.78
  - Desviación típica: 807.43
- Clase talk.politics.guns:
  - Mínimo: 68
  - Máximo: 3869
  - Media: 415.33

    - Desviación típica: 473.05
- Clase talk.politics.mideast:
    - Mínimo: 55
    - Máximo: 10385
    - Media: 727.88
    - Desviación típica: 1321.51

Para calcular estos valores se le ha aplicado un pequeño preprocesado: se ha sustituido todos aquellos caracteres que representaban espacios en blanco usando la expresión regular "\s+" por espacios en blanco " " (Unicode U+0020), de forma que al hacer el conteo de caracteres se separase las palabras usando la expresión regular " +" (espacio en blanco al menos una vez). Este preprocesado no se mantiene cuando se realiza el preprocesado como tal.

A partir de la clasificación por carpetas se forma el gold standard en el código. El gold standard será representado por un diccionario donde las claves serán los nombres de los documentos y los valores asociados a cada clave la clase (carpeta) a la que pertenecen.

Definición del preprocesado

El preprocesamiento de los documentos consistirá en 5 fases:

1. Se elimina todo aquello del documento que no sea útil para la representación. Se eliminará:
    a. **Cabeceras**. Están formadas por n líneas consecutivas al comienzo del documento y separadas del cuerpo del email por, al menos, una línea con un carácter \n.
    b. **Firmas**. A pesar de la irregularidad, la gran mayoría están al final del documento, separadas explícitamente del cuerpo del email usando filas de caracteres "-", "_", "=" o "+" y ocupan unas pocas líneas. Por ello, para detectar las firmas se usa la expresión regular "^[\t]*[-+=_]+[\t\n\r]*$" para detectar el comienzo de una posible firma. Si a partir de esa línea se tienen como mucho 9 líneas más (incluyendo la encontrada), se toma como firma ese conjunto de líneas.
    c. **Líneas en blanco**. Se eliminan todas las líneas que no contenga ningún carácter diferente a los espacios en blanco.
    d. **Caracteres en blanco.** Se usa la expresión regular "\s+" para transformar todos los caracteres en blanco que queden por el resto del texto en espacios en blanco (carácter Unicode U+0020).
    e. **Correos electrónicos.** Usando la expresión regular "\w+(?:\.\w+)*@\w+(?:\.\w+)+" elimina los correos electrónicos.

    f. **Signos de puntuación.** Usando los signos de puntuación definidos en `string.punctuation`.
    g. **Números.** Todos los números que aparezcan usando la expresión "\d+(?:[\.,]\d+)?". Por definición del pipeline, la segunda parte de la expresión no sería necesaria por la eliminación de los signos de puntuación en el paso anterior, pero se mantiene por conveniencia.
2. Se transforma todo el texto a minúsculas para tenerlo normalizado.
3. Se realiza el tokenizado del texto. Se divide según los espacios en blanco que haya, usando la expresión regular " +".
4. Se eliminan las palaras vacías usando el corpus del modelo en_core_web_lg de la librería spaCy.
5. Se transforman las palabras que quedan en sus raíces usando el stemmer de Porter.

Durante el proceso de preprocesado, antes de obtener las raíces de las palabras mediante el stemmer, se almacena el texto preprocesado en una carpeta denominada corpus-clustering-limpio.

Funciones de pesado

Para las funciones de pesado se ha optado por una función local, la normalización logarítmica, y una función global, el TF-IDF.

La representación de los documentos según la normalización logarítmica se encuentra en el DataFrame `matriz_norm_log`, donde las filas son los documentos y las columnas los rasgos.

La representación de los documentos según el TF-IDF se encuentra en el DataFrame `matriz_tf_idf`, con la misma estructura que la matriz de la otra representación.

Previo a la generación de estas matrices, se eliminan aquellos rasgos que no aparecen en, al menos, 7 documentos del total de documentos del corpus.

Estas matrices quedan con 1904 rasgos y 805 documentos. Son matrices dispersas debido a la abundancia de ceros: con un total de 1.532.720 posiciones, 1.489.060 de ellas son con valor cero, lo que representa más del 97% de la matriz.

Algoritmos de clustering

Además del algoritmo K-means indicado en la práctica, también ser hará uso del algoritmo LDA. La elección viene en parte motivada por uno de los usos posibles de este algoritmo: la reducción de la dimensionalidad de las entradas.

Los resultados de los algoritmos quedan almacenados en un diccionario con la siguiente estructura:

```
{
    'kmeans' : {
        'log' : {...},
        'tf_idf' : {...}
    },
    'lda' : {
        'log' : {...},
        'tf_idf' : {...}
    }
}
```

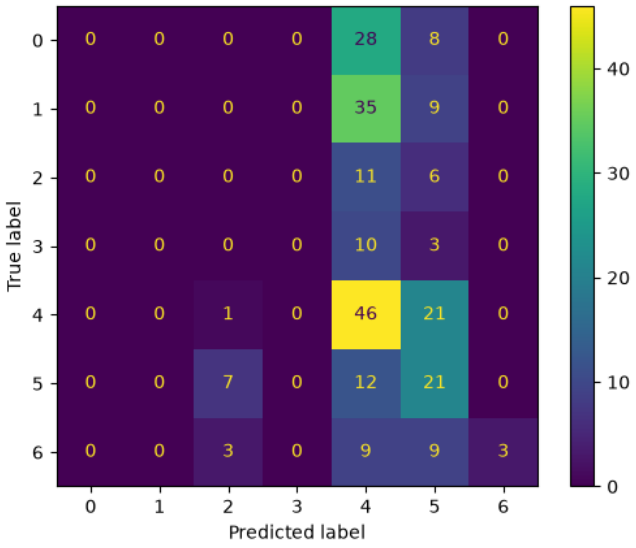
Previo a la utilización de los algoritmos, se generan los dos conjuntos de entrenamiento y de prueba, con 70% y 30% de los datos respectivamente. A partir del conjunto de entrenamiento se ajusta un escalador robusto y se escalan ambos subconjuntos de datos.

Para el algoritmo K-means es necesario establecer una correlación entre las etiquetas reales y los clústeres en los que son clasificados los documentos. Para realizar el mapeo óptimo se usa el algoritmo húngaro mediante la función `linear_sum_assignment`. Este algoritmo no proporciona la combinación entre pares de etiquetas de forma que para cada equivalencia se tiene el número máximo de documentos.

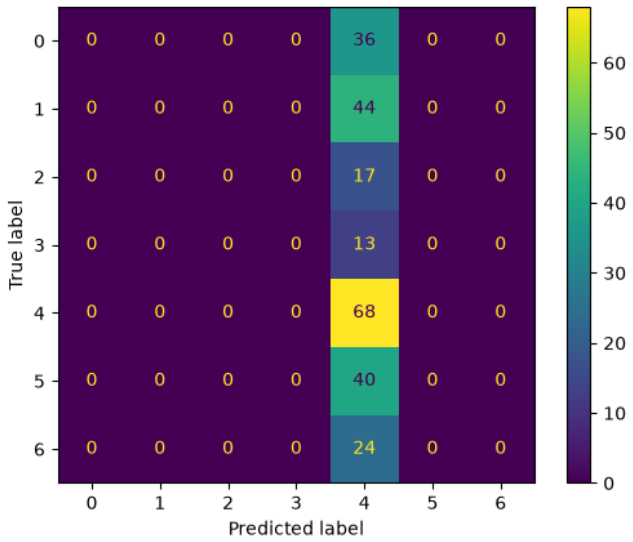
Resultados y conclusiones

Para el algoritmo K-means se ha obtenido:

- Usando la normalización logarítmica:
    - Precisión: 0,22
    - Recall: 0,189
    - F1-score: 0,143



- Usando la TF-IDF:
    - Precisión: 0,04
    - Recall: 0,14
    - F1-score: 0,06

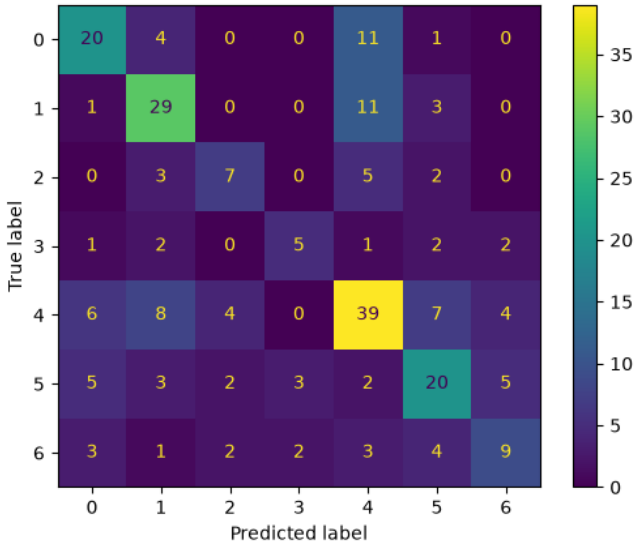


En el caso del K-means podemos ver como tiene predilección a agrupar todos los documentos como si fueran de la clase 4. En el caso del TF-IDF es mucho más acentuado esta agrupación, aunque en ambos casos se ofrecen resultados muy pobres.

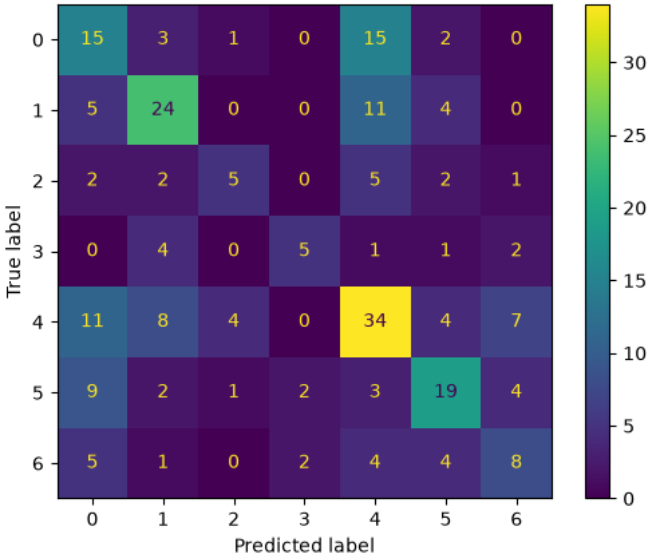
A partir de estos resultados es bastante complicado sacar una conclusión acerca de la dificultad de agrupar los documentos por clase debido a la poca variabilidad en la predicción.

Por otra parte, para el LDA, se han obtenido los siguientes resultados:

- Usando normalización logarítmica:
    - Precisión: 0,515
    - Recall: 0,49
    - F1-score: 0,50



- Usando el TF-IDF:
  - Precisión: 0,46
  - Recall: 0,42
  - F1-score: 0,435



En este caso, el algoritmo vuelve a tener predilección por la clase 4, y se puede ver más de manera clara como se confunde especialmente con las dos primeras clases, quizás por el hecho de ser sobre hardware (clases 0 y 1) y la 4 ser sobre electrónica. La clase que más le cuesta identificar es la 2, 3 y 6, que las confunde de forma relativamente uniforme con el resto de las clases.